Engenharia de Características para Aprendizagem Computacional / Engenharia de Atributos

Application Domain: Text

Licenciatura em Enheraria Informática / Licenciatura em Inteligência Artificial e Ciência de Dados / Licenciatura em Engenharia e Ciência de Dados
Edition 2025-26

Marco Simões

Text representation

Text must be transformed into numbers/arrays so that computers can process them

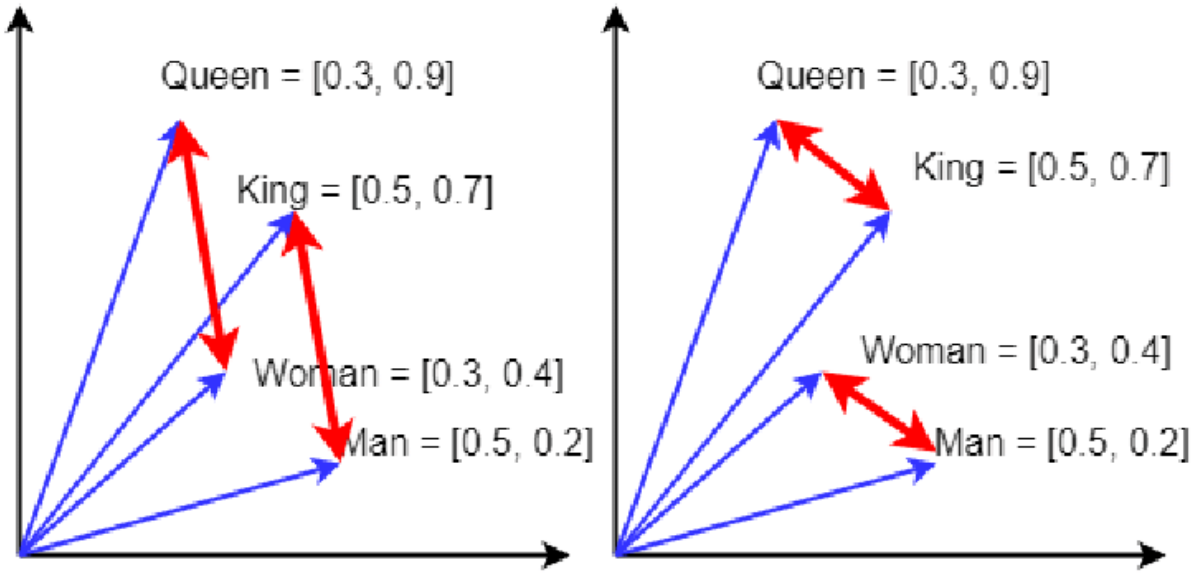
Nomenclature:

Corpus: a **corpus** (plural: **corpora**) is a **collection of text data** used for training, testing, or analyzing language models.

-

Term	Definition	Example
Token	A single word or character	`"hello"`
Sentence	A sequence of tokens	`"This is an NLP example."`
Document	A collection of sentences	`"This is document one. It has multiple sentences."`
Corpus	A collection of documents	`[doc1, doc2, doc3]`

Vector representation of words



$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}},$$

Bag of Words (BOW)

Bag of Words Model explanation

Corpus
A collection of text documents

↓

Tokenize
Divide the text into smaller units called tokens, usually words or phrases

↓

Count word frequencies
Create a vocabulary of unique words and count the number of times each word appears in each document.

↓

Encode the data
Encoding the text data as numerical values by creating a vector for each document, with each element of the vector representing the frequency count of a particular word in the document

© AIML.com Research

Example

Corpus:

The dog is happy. The child makes the dog happy. The dog makes the child happy

↓

Tokenization:

D1: [The] [dog] [is] [happy]
D2: [The] [child] [makes] [the] [dog] [happy]
D3: [The] [dog] [makes] [the] [child] [happy]

↓

Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

↓

Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

Title: Bag of Words model explained using an example
Source: AIML.com Research

Bag of Words (BOW)

- Represents text as a **vector of word occurrences**.
- Ignores word order but considers **word frequency**.

✅ **Python Example:**

python Copiar Editar

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = ["This is a good song", "This is a bad song", "I love this song"]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print("Feature Names:", vectorizer.get_feature_names_out())
print("Feature Matrix:\n", X.toarray())
```

✅ **Pros:** Simple, effective for short texts.

❌ **Cons:** High-dimensional, ignores word meaning.

SPARCITY

Most documents will typically use a **very small subset of the words** used in the **corpus**

Resulting matrix will have many feature values that are zeros
(typically more than 99% of them)

Example:

A collection of 10,000 short text documents (such as emails) will use a vocabulary with a size in the order of 100,000 unique words in total.

In contrast, each document will use 100 to 1000 unique words individually.

Sparse Matrix

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 2 \\ 9 & 0 & 0 & 6 \\ 7 & 0 & 0 & 0 \end{bmatrix} \rightarrow$$

Row	Column	Value
0	1	1
2	1	5
2	3	2
3	0	9
3	3	6
4	0	7

Term Frequency - Inverse Document Frequency (TF-IDF)

- Balance between importance of the term (frequency) in the document (more frequent) and the frequency of the term in the corpus

$$\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

$$\mathrm{idf}(t,D) = \log \frac{N}{|\{d : d \in D \text{ and } t \in d\}|}$$

$$\mathrm{tfidf}(t,d,D) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t,D)$$

Term Frequency - Inverse Document Frequency (TF-IDF)

- Weighs words by **importance in a document**.
- Gives less weight to common words (e.g., "the", "is").

✅ **Python Example:**

python Copiar Editar

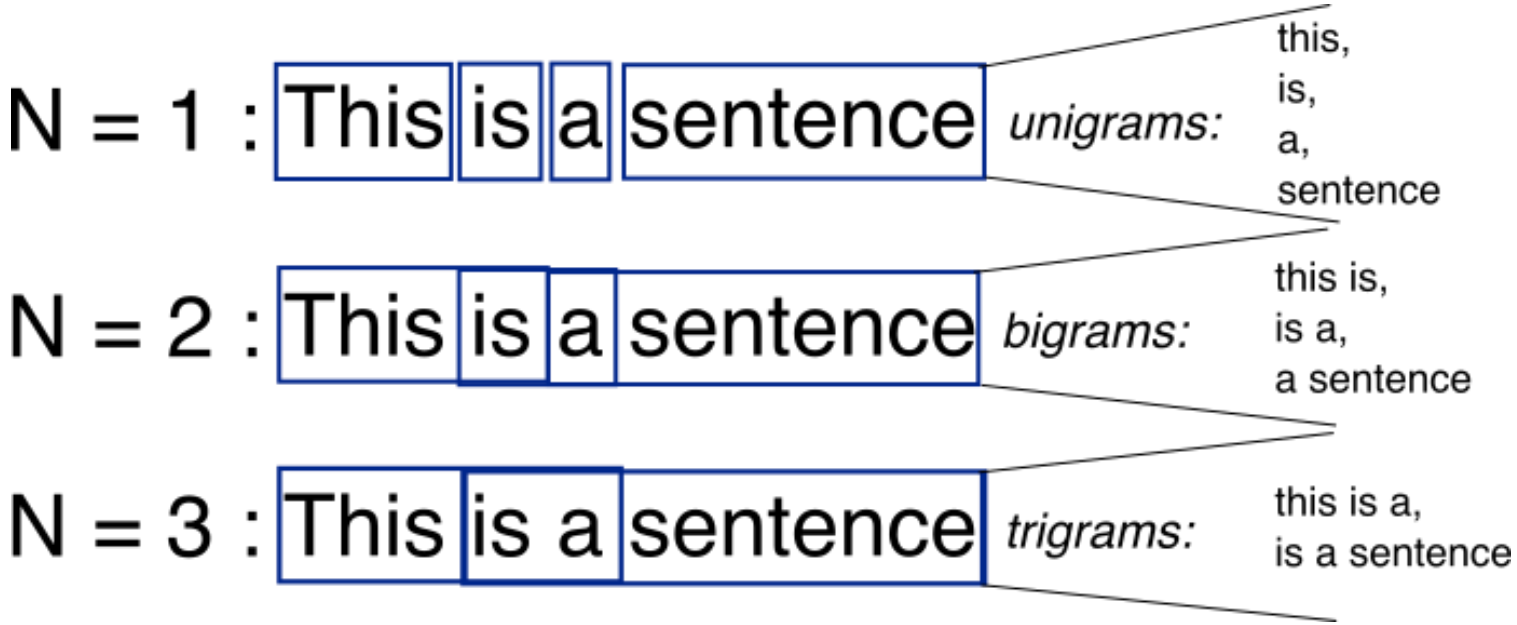
```python
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print("Feature Names:", vectorizer.get_feature_names_out())
print("TF-IDF Matrix:\n", X.toarray())
```

✅ **Pros:** Reduces importance of frequent words, effective for classification.

❌ **Cons:** Still ignores context and word meaning.

N-GRAMS (Bi-GRAMS, Tri-GRAMS)

Extend previous methods to consider sequences of tokens



N-GRAMS (Bi-GRAMS, Tri-GRAMS)

- Extends BoW/TF-IDF by considering **sequences of words** (e.g., bigrams, trigrams).
- Captures **phrases and word dependencies**.

✅ **Python Example:**

python Copiar Editar

```python
vectorizer = CountVectorizer(ngram_range=(1, 2))  # Unigrams + Bigrams
X = vectorizer.fit_transform(corpus)
print("N-Gram Feature Names:", vectorizer.get_feature_names_out())
```

✅ **Pros:** Preserves some context, useful for **sentiment analysis**.

❌ **Cons:** Can create **very large feature spaces**.

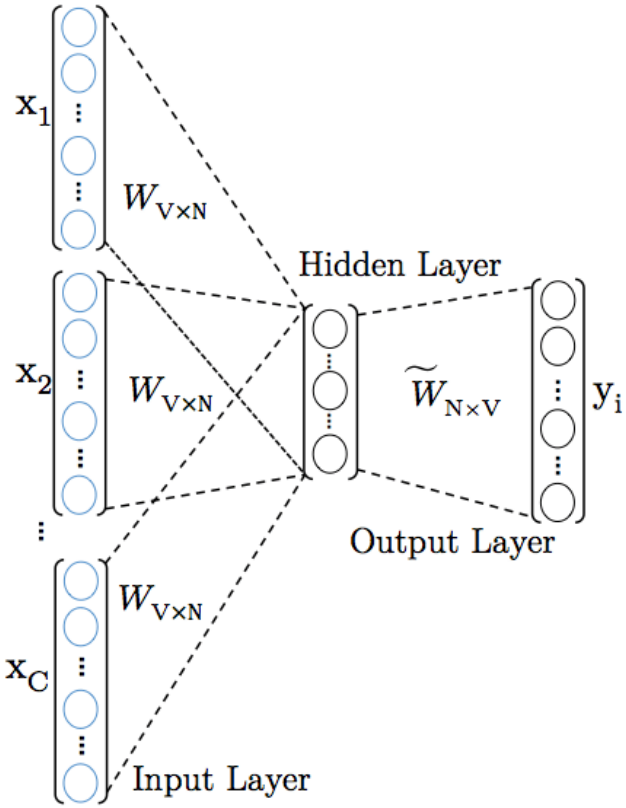
Word2Vec

Context added to generate the embedding

Words that appear in similar contexts will have similar values

Example:

The **kid** likes to play football
The **child** likes to play football



Word2Vec

```
from gensim.models import Word2Vec

sentences = [sentence.split() for sentence in X_train]
w2v_model = Word2Vec(sentences, size=100, window=5, min_count=5, workers=4)
```

```
def vectorize(sentence):
    words = sentence.split()
    words_vecs = [w2v_model.wv[word] for word in words if word in w2v_model.wv]
    if len(words_vecs) == 0:
        return np.zeros(100)
    words_vecs = np.array(words_vecs)
    return words_vecs.mean(axis=0)
```

Extract sentiment from text

Polatiry scores:

How positive or negative is each word?

(A) VADER (Valence Aware Dictionary and sEntiment Reasoner)

✅ **VADER is specialized for short, informal text (e.g., tweets, reviews).**

✅ **It computes four scores**:

- **pos (positive)** → Sum of positive word scores.
- **neg (negative)** → Sum of negative word scores.
- **neu (neutral)** → Proportion of words that are neutral.
- **compound** → Overall polarity score **(final sentiment value)**.

✅ **Formula for** `compound` **score** (normalized sum of word scores):

$$\text{compound} = \frac{\sum \text{word sentiment scores}}{\sqrt{\sum \text{word scores}^2 + \alpha}}$$

(where **α** is a normalization factor)

Extract sentiment from text

✅ **Example: Computing Polarity Scores with VADER**

python Copiar Editar

```python
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

nltk.download('vader_lexicon')
sia = SentimentIntensityAnalyzer()

text = "I absolutely love this song! It's amazing and makes me happy."
sentiment_scores = sia.polarity_scores(text)

print("Sentiment Scores:", sentiment_scores)
```

🔷 **Output:**

plaintext Copiar Editar

```plaintext
{'neg': 0.0, 'neu': 0.294, 'pos': 0.706, 'compound': 0.8516}
```

✅ `compound` **score interpretation:**

- **Positive (>0.05)** → Positive sentiment
- **Negative (<-0.05)** → Negative sentiment
- **Neutral (-0.05 to 0.05)** → Neutral sentiment

Extract sentiment from text

(B) SentiWordNet

✅ SentiWordNet assigns three scores to each word:

- PosScore (Positive Sentiment)
- NegScore (Negative Sentiment)
- ObjScore (Objectivity, Neutrality)

✅ Formula for Final Score:

$$\text{Sentiment Score} = \text{PosScore} - \text{NegScore}$$

Extract sentiment from text

✅ **Example: Computing Sentiment Score from SentiWordNet**

python Copiar Editar

```python
from nltk.corpus import sentiwordnet as swn
from nltk import word_tokenize, pos_tag
import nltk

nltk.download('sentiwordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt')

# Example word sentiment lookup
synset = list(swn.senti_synsets('happy', 'a'))[0]  # 'a' for adjective
print(f"Positive: {synset.pos_score()}, Negative: {synset.neg_score()}, Objective:
```

🔷 **Output Example:**

plaintext Copiar Editar

```plaintext
Positive: 0.75, Negative: 0.0, Objective: 0.25
```

✅ **Final Sentiment Score: 0.75 (Positive)**

Extract sentiment from text

(C) AFINN (Finn Årup Nielsen's Sentiment Lexicon)

✅ **AFINN assigns integer sentiment scores (-5 to +5) to words.** ✅ **Polarity score is simply the sum of word scores.**

✅ **Example: Computing Sentiment Score with AFINN**

python Copiar Editar

```python
from afinn import Afinn

afinn = Afinn()
text = "This song is absolutely fantastic and uplifting!"
sentiment_score = afinn.score(text)

print("AFINN Sentiment Score:", sentiment_score)
```

🔷 **Output Example:**

plaintext Copiar Editar

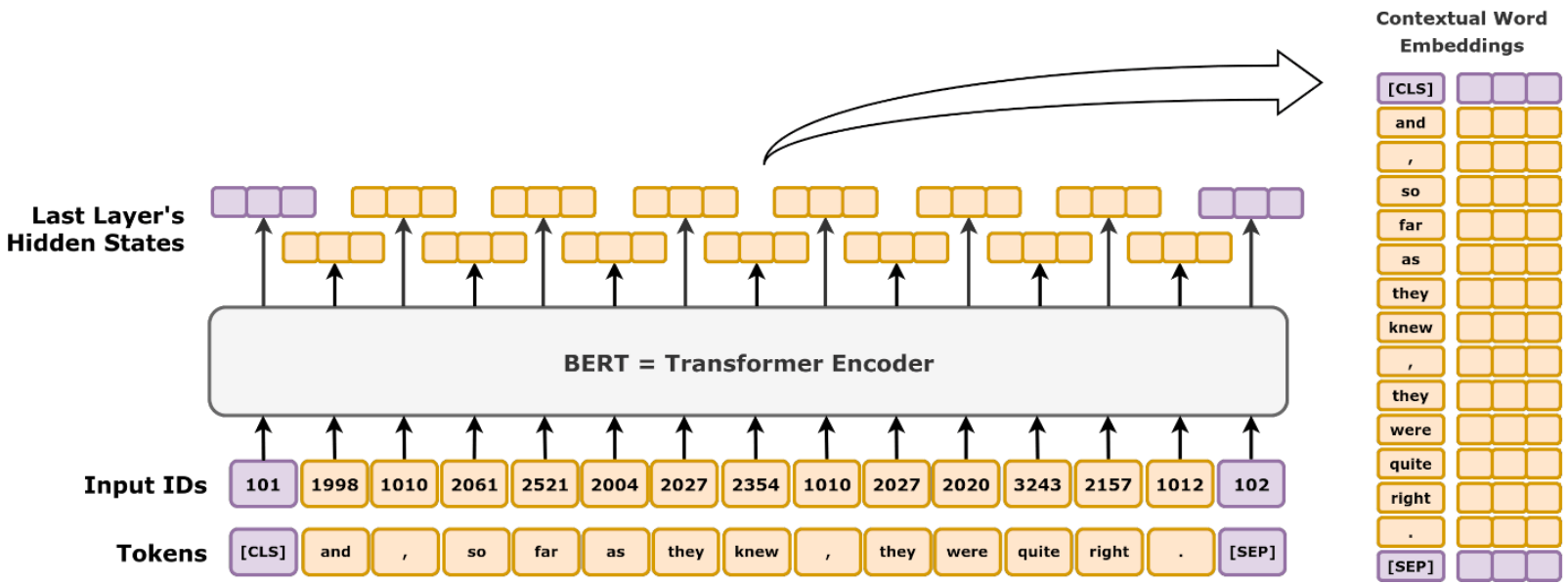
```plaintext
AFINN Sentiment Score: 7
```

✅ **Score Interpretation:**

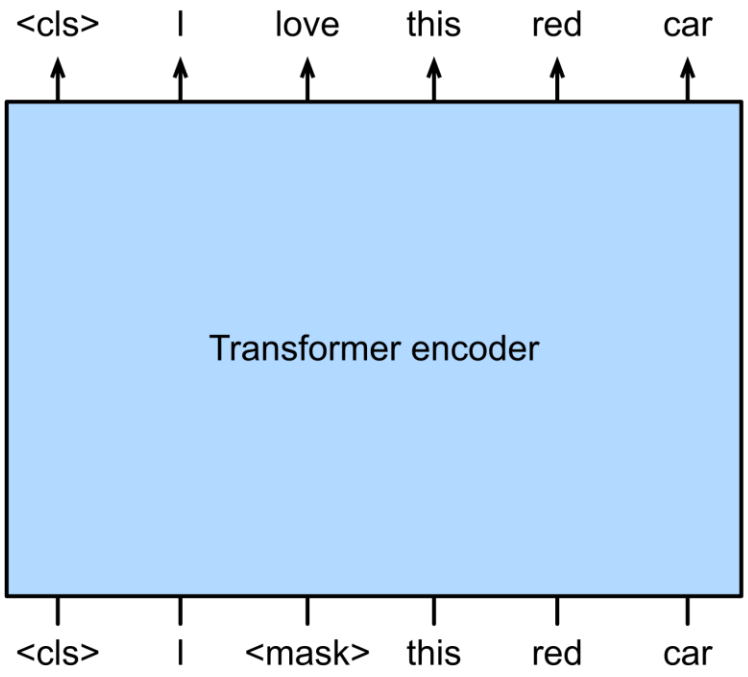
- **Positive (> 0)** → Positive sentiment
- **Negative (< 0)** → Negative sentiment
- **Neutral (0)** → Neutral sentiment

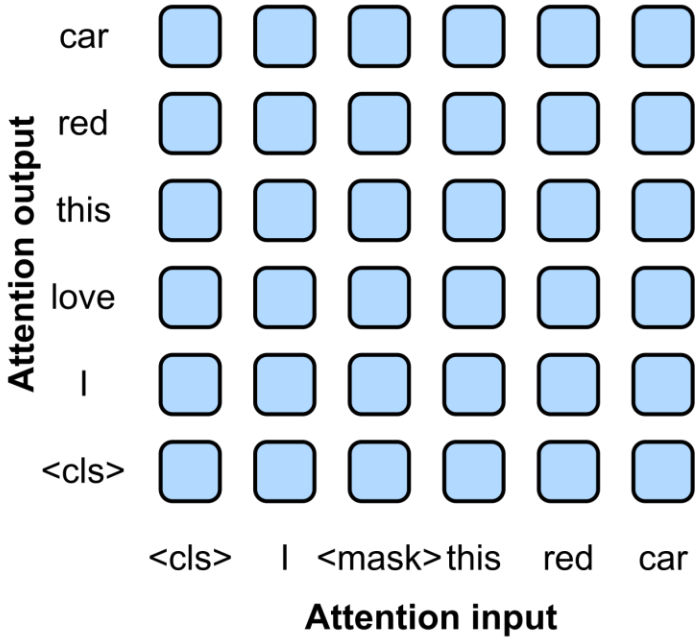
LARGE LANGUAGE MODELS

Bidirectional encoder representations from transformers (BERT)



Bidirectional encoder representations from transformers (BERT)





Bidirectional encoder representations from transformers (BERT)

```python
from transformers import BertTokenizer, BertModel
import torch

# Load Pretrained BERT Model
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

# Tokenize and Extract Features
def extract_text_features(sentence):
    """Extracts text embeddings from the last layer of a BERT model."""

    tokens = tokenizer(sentence, return_tensors="pt", padding=True, truncation=True)

    with torch.no_grad():
        outputs = model(**tokens)


    # Get CLS token embedding (First Token)
    embedding = outputs.last_hidden_state[:, 0, :].squeeze()

    # Get Sentence Embedding (Mean Pooling of Last Layer)
    #embedding = outputs.last_hidden_state.mean(dim=1).squeeze()
    return embedding
```